Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа

по курсу «Численные методы»

«Решение СЛАУ с 3-х диагональной матрицей методом прогонки»

Студент группы ИУ9-62Б: Трофименко Д. И.

Преподаватель: Домрачева А. Б.

Москва 2026

1 Постановка задачи

1.1 Дано:

Система линейных алгебраических уравнений (СЛАУ) с 3-х диагональной матрицей

$$A\bar{x} = \bar{d}, \tag{1}$$

где $A \in \mathbb{R}^{n\times n}$ — трёхдиагональная матрица, $\bar{d} \in \mathbb{R}^n$ — вектор правой части, $\bar{x} \in \mathbb{R}^n$ — вектор неизвестных, $l = 2s+1$ — количество диагоналей, s — ширина ленты.

Трёхдиагональная матрица имеет вид

$$A = \begin{pmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_1 & b_2 & c_2 & \ddots & \vdots \\ 0 & a_2 & b_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{n-1} \\ 0 & \cdots & 0 & a_{n-1} & b_n \end{pmatrix}.$$

1.2 Найти:

$\bar{x} \in \mathbb{R}^n$ — ?

2 Теоретические сведения

2.1 Трёхдиагональная СЛАУ

Для трёхдиагональной матрицы система $Ax = d$ эквивалентна набору уравнений:

$$\begin{cases} b_1x_1 + c_1x_2 & = d_1, \\ a_1x_1 + b_2x_2 + c_2x_3 & = d_2, \\ \vdots & \\ a_{i-1}x_{i-1} + b_ix_i + c_ix_{i+1} & = d_i, \\ \vdots & \\ a_{n-1}x_{n-1} + b_nx_n & = d_n. \end{cases} \tag{2}$$

Ширина ленты $s = 1$, количество диагоналей $l = 2s + 1 = 3$.

2.2 Метод прогонки

Будем выражать каждую неизвестную через следующую:

$$x_i = \alpha_i x_{i+1} + \beta_i, \qquad i = 1, \ldots, n-1. \tag{3}$$

Коэффициенты α_i и β_i находятся последовательно из уравнений системы.

Прямой ход. Из первого уравнения системы

$$b_1x_1 + c_1x_2 = d_1$$

выражаем x_1 через x_2:

$$x_1 = -\frac{c_1}{b_1}\,x_2 + \frac{d_1}{b_1}, \qquad b_1 \neq 0. \tag{4}$$

Сравнивая (4) с видом (3), получаем начальные коэффициенты:

$$\alpha_1 = -\frac{c_1}{b_1}, \qquad \beta_1 = \frac{d_1}{b_1}, \qquad b_1 \neq 0.$$

Далее подставляем уже найденное представление $x_{i-1} = \alpha_{i-1}x_i + \beta_{i-1}$ в i-е уравнение системы

$$a_{i-1}x_{i-1} + b_i x_i + c_i x_{i+1} = d_i.$$

После подстановки имеем

$$a_{i-1}(\alpha_{i-1}x_i + \beta_{i-1}) + b_i x_i + c_i x_{i+1} = d_i,$$

то есть

$$(a_{i-1}\alpha_{i-1} + b_i)x_i + c_i x_{i+1} = d_i - a_{i-1}\beta_{i-1}.$$

Отсюда выражаем x_i через x_{i+1}:

$$x_i = -\frac{c_i}{a_{i-1}\alpha_{i-1} + b_i}\, x_{i+1} + \frac{d_i - a_{i-1}\beta_{i-1}}{a_{i-1}\alpha_{i-1} + b_i}, \qquad a_{i-1}\alpha_{i-1} + b_i \neq 0. \tag{5}$$

Снова сравнивая (5) с (3), получаем формулы прямого хода:

$$\alpha_i = -\frac{c_i}{a_{i-1}\alpha_{i-1} + b_i}, \qquad i = 2, \ldots, n-1, \tag{6}$$

$$\beta_i = \frac{d_i - a_{i-1}\beta_{i-1}}{a_{i-1}\alpha_{i-1} + b_i}, \qquad i = 2, \ldots, n. \tag{7}$$

$a_{i-1}\alpha_{i-1} + b_i$ является знаменателем, поэтому при равенстве его нулю дальнейшие вычисления невозможны из-за деления на ноль.

Обратный ход. После завершения прямого хода из последнего уравнения получается

$$x_n = \beta_n. \tag{8}$$

Затем последовательно восстанавливаем остальные компоненты, двигаясь справа налево:

$$x_i = \alpha_i x_{i+1} + \beta_i, \qquad i = n-1, n-2, \ldots, 1. \tag{9}$$

3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1 — lab.py.

```
import numpy as np

A = [[4, 1, 0, 0],
     [1, 4, 1, 0],
     [0, 1, 4, 1],
     [0, 0, 1, 4]]

d = [5, 6, 6, 5]

def diagonal_check(A):
    n = len(A)
    for i in range(1, n):
        if A[i][i] == 0:
            print('The elements of the main diagonal are equal to 0')
            return False
    return True

def find_vector_x(A, d):
    n = len(A)

    if (not diagonal_check(A)):
        return -1

    x = [0 for k in range(0, n)]

    alpha = [0 for k in range(0, n)]
    beta = [0 for k in range(0, n)]
    f = [0 for k in range(0, n)]

    f[0] = A[0][0]
    alpha[0] = - A[0][1] / f[0]
    beta[0] = d[0] / f[0]

    for i in range(1, n-1):
        f[i] = A[i][i] + A[i][i-1]*alpha[i-1]
        alpha[i] = - A[i][i+1] / f[i]
        beta[i] = (d[i]-A[i][i-1]*beta[i-1]) / f[i]

    f[n-1] = A[n-1][n-1] + A[n-1][n-2] * alpha[n-2]
    beta[n-1] = (d[n-1] - A[n-1][n-2] * beta[n-2]) / f[n-1]

    x[n-1] = beta[n-1]
    for i in range(n-1, 0, -1):
        x[i-1] = alpha[i-1] * x[i] + beta[i-1]

    for i in range(0, n):
        print(f'x[{i}] =', x[i])
    print()

find_vector_x(A, d)
```

3.1 Результаты

В качестве исходных данных задаются следующие значения:

$$A = \begin{pmatrix} 4 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 4 \end{pmatrix}, \qquad d = \begin{pmatrix} 5 \\ 6 \\ 6 \\ 5 \end{pmatrix}.$$

При выполнении программы получено следующее решение:

$$x = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

Вывод программы:

x[0] = 1.0
x[1] = 1.0
x[2] = 1.0
x[3] = 1.0

4 Выводы

В работе реализован метод прогонки для численного решения СЛАУ с трёхдиагональной матрицей, который за счёт прямого вычисления вспомогательных коэффициентов и последующего обратного восстановления неизвестных обеспечивает эффективное получение решения. В рамках поставленной задачи алгоритм корректно обрабатывает заданные входные данные, предотвращая вычислительные ошибки за счёт контроля недопустимых ситуаций (нулевых элементов на главной диагонали и возможного деления на ноль через проверку знаменателей), и обеспечивает получение решения с точностью, ограниченной лишь погрешностями машинного округления при вычислениях с плавающей точкой.